



UADY
UNIVERSIDAD
AUTÓNOMA
DE YUCATÁN

Facultad de matemáticas

Licenciatura En ingeniería En Software

programación Orientada A Objetos

Profesor:

Edgar Antonio Cambranes Martínez

Integrantes:

Juan Manuel Hernandez Miranda

Leonardo Lomas

Rolando Cabrera

Alexander Castañeda

Paola Parra

Freddy Uitzil

propósito:

documentación del diagrama de clases

documentación del diagrama de clases:

Advertencias principales

tenemos en principio que saber que esta es el diagrama 2.0, ya que antes de este hubo unos cuantos prototipos por lo que ahora podemos decir que este es el producto final del diagrama o el que más se acerca a la representación conceptual del proyecto.

propósito

tener muy claras las principales clases en el programa para de esta manera a la hora de empezar a codificar se cometan los menores errores posibles, ya que todo el equipo sabrá a que dirección hay que ir para lograr el proyecto, y poder usar el patrón de diseño MVC para de esta manera tener una mejor estructura en el diagrama de clases.

Explicación de las clases usadas

Para que haya un mejor entendimiento de las clases no solo para nosotros (integrantes del proyecto), si no también para personas externas a este proyecto, hare una explicación detallada de las clases intentando ser lo mas claro y sin complicar tanto las cosas.

MODELOS (Model)

BaseModel

- Atributos: id, createdAt, updatedAt.
- Métodos: `validate()`, `save()`, `delete()`.
- Responsabilidad: lógica común de persistencia/validación; **no** maneja UI.

PatientModel

- Datos: nombre, apellidos, fecha de nacimiento, contacto, referencia a `Record`.
- Métodos clave: `getAppointments()` (obtiene citas asociadas), `addRecordEntry(entry)`.
- Responsabilidad: representar al paciente y operaciones de negocio simples (añadir entrada de expediente).

TherapistModel

- Datos: nombre, especialidades, `Schedule` (estructura con disponibilidad).
- Métodos: `isAvailable(start,end)`, `getAppointments()`.
- Responsabilidad: proveer consultas sobre la disponibilidad y sus datos.

RoomModel

- Datos: nombre, capacidad.
- Métodos: `isAvailable(start,end)`.
- Responsabilidad: validar disponibilidad física de sala.

AppointmentModel

- Datos: referencias a `patientId`, `therapistId`, `roomId`, `startTime`, `endTime`, `status`.
- Métodos: `schedule()`, `reschedule()`, `cancel()`, `conflictsWith(...)`.
- Responsabilidad: lógica de negocio de una cita (estado, reglas de conflicto).

RecordModel

- Datos: lista de `RecordEntry` (nota clínica, fecha, autor).
- Métodos: `addEntry`, `getEntries`.
- Responsabilidad: almacenar historial clínico del paciente.

PaymentModel

- Datos: monto, estado, referencia a cita.
- Métodos: `processPayment()`.
- Responsabilidad: registrar y procesar pagos asociados a citas.

UserModel

- Datos: username, passwordHash, role.
- Métodos: `authenticate()`, `authorize()`.
- Responsabilidad: seguridad/autenticación y comprobación de roles.

ReportModel

- Métodos: `generateAppointmentsReport`, `generateFinancialReport`.
- Responsabilidad: encapsular la creación de reportes a partir de datos.

VISTAS (View)

Las vistas son “plantillas” / componentes UI. Sólo muestran datos y envían eventos al controlador:

BaseView

- `render(data)` y `bindEvents()`.

AppointmentView

- Muestra calendario, lista de citas, modal para ver/crear/editar cita.
- Envía eventos al `AppointmentController` (ej. cuando el usuario envía formulario de nueva cita).

PatientView, TherapistView, RoomView, RecordView, AuthView, DashboardView, ReportView

- Roles análogos: renderizar sus datos, formularios y emitir eventos (create/update/search/export).
- Responsabilidad: UI/UX y validación ligera (cliente).

CONTROLADORES (Controller)

Son la capa que recibe eventos desde las Vistas, coordina Modelos/Repositorios/Servicios y responde actualizando Vistas.

BaseController

- Lógica común de inicialización.

AppointmentController

- Métodos: `createAppointment(data)`, `updateAppointment(...)`, `cancelAppointment(...)`, `listAppointments(filter)`.
- Flujo típico en `createAppointment`:
 1. Validar datos (fechas, paciente, terapeuta).
 2. Consultar `AppointmentRepository.findConflicts(...)` y `TherapistModel.isAvailable(...)` y `RoomModel.isAvailable(...)`.
 3. Si OK → `AppointmentModel.schedule()` y `AppointmentRepository.save()`.
 4. Notificar via `NotificationService.sendEmail(...)`.
 5. Actualizar `AppointmentView.renderCalendar(...)`.
- Responsabilidad: reglas de negocio para citas.

PatientController, TherapistController, RoomController

- CRUD y validaciones específicas de cada entidad.

RecordController

- Añadir entradas al expediente, bloquear acceso según rol.

AuthController

- Login/logout, tokens, verificación de permisos usando **AuthService** y **UserModel**.

ReportController

- Llama a **ReportModel.generate...** y pasa datos a **ReportView** para exportar.

REPOSITORIOS / SERVICIOS AUXILIARES

AppointmentRepository

- **Responsabilidad:** Encapsula consultas DB complejas: buscar conflictos, listar por terapeuta/paciente, persistencia.
- **Actividades:** índices por horario, locking/optimistic locking para evitar race conditions.
- **Métodos:** save(appointment), update(appointment), delete(id)

NotificationService

- **Responsabilidad:** Envía correos/SMS confirmaciones y recordatorios.
- **Actividades:** notificar al usuario, notificar a los administradores, notificar a los terapeutas

AuthService

- **Responsabilidad:** Hash de contraseñas, verificación, generación de tokens de sesión/JWT.
- **Actividades:** autorizar al usuario, verificar el nivel de prioridad
- **Métodos:** hashPassword(pw) : String, verifyPassword(hash, pw) : bool, generateToken(user) : String

PatientService

- **Responsabilidad:** CRUD paciente, validaciones de integridad (DNI/curp), sincronización con record.
- **Actividades:** crear paciente, actualizar datos, buscar por filtros, verificar consentimientos.
- **Métodos:** create(dto) : PatientModel, findById(id), update(id,dto)

TherapistService

- **Responsabilidad:** gestionar disponibilidad, especialidades, asignaciones.
- **Actividades:** disponibilidad calendarizada, bloqueo por vacaciones, carga horaria.
- **Métodos:** isAvailable(id,start,end), assignSchedule(id, schedule)

RoomService

- **Responsabilidad:** gestión de salas y su disponibilidad (incluye restricciones de capacidad).
- **Actividades:** reservar sala, verificar solapamientos, mantenimiento.
- **Métodos:** isAvailable(id,start,end), reserveRoom(id, start,end)

RecordService

- **Responsabilidad:** lógica sobre expedientes clínicos (normas de privacidad y versionado).
- **Actividades:** añadir entrada, validar confidencialidad, controlar acceso por rol, almacenar metadatos (autor, timestamp).
- **Métodos:** addEntry(patientId, entry, author) : RecordEntry, getRecord(patientId)

AuthService

- **Responsabilidad:** autenticar usuarios y verificar autorizaciones.
- **Actividades:** hashing de contraseñas, generación/validación de tokens (JWT), verificación de roles y permisos finos (p. ej. quién puede ver expedientes).
- **Métodos:** hashPassword(pw), verifyPassword(hash,pw), generateToken(user), hasRole(user, role)

ReportService

- **Responsabilidad:** generar reportes agregados y exportables (PDF/CSV).
- **Actividades:** consultas analíticas sobre repositorios (citas por periodo, ingresos, ocupación), paginación, formato/export.
- **Métodos:** generateAppointmentsReport(start,end), generateFinancialReport(start,end)

NotificationService

- **Responsabilidad:** enviar notificaciones (correo, SMS, push).
- **Actividades:** plantillas, colas de envío (async), reintentos, logs de entrega.
- **Métodos:** sendEmail(to,subject,body), sendSMS(to,message), enqueueNotification(event)

Estos servicios se inyectan en controladores; **no** pertenecen ni a Vista ni a Modelo (son helpers / infra), probablemente haya mas service que se usaran en el diagrama, pero tiene usos parecidos a los tres que hemos mencionado, ya que lo que hacen es toda la lógica para programar, re-programar y validar citas.

Relaciones entre clases y multiplicidades (explicado)

1. Asociaciones principales

- **AppointmentModel** → contiene/usa: 1 **PatientModel**, 1 **TherapistModel**, 1 **RoomModel**.
- **PatientModel** 1 — * **AppointmentModel** (un paciente puede tener muchas citas).
- **TherapistModel** 1 — * **AppointmentModel** (un terapeuta atiende muchas citas).
- **RoomModel** 1 — * **AppointmentModel** (una sala puede tener muchas citas).
- **PatientModel** 1 — 1 **RecordModel** (cada paciente tiene su expediente).

2. Controladores ↔ Vistas

- Las **Vistas llaman** métodos del Controller cuando el usuario hace acciones (ej. envía formulario).
- Los **Controllers actualizan** las Vistas llamando **render(data)** o emitiendo eventos (actualización UI).
- Relación típica: **AppointmentView** → **AppointmentController** (evento create), **AppointmentController** → **AppointmentView** (actualizar calendario).

3. Controllers ↔ Models / Repos

1) Herencia (BaseController <|-- XController)

Todas las clases controller extienden BaseController para compartir:

- inicialización común (init(models, views, services)), manejo estándar de errores, logging y utilidades (paginación, validaciones comunes).
- **Multiplicidad:** una relación 1:1 (cada controller extiende una sola base).

Por qué: evita duplicación de código y centraliza comportamiento cross-cutting.

2) MainController → otros controllers

MainController "1" -- "*" AppointmentController (y Patient/Report):

- MainController **orquesta** la inicialización y la interacción entre varios controllers cuando la UI necesita coordinar varias vistas (p. ej. Dashboard que muestra citas + pacientes + reportes).
- **Multiplicidad:** 1 MainController compone múltiples controllers (1 → *).

Por qué: centraliza la navegación y coordinación sin que controllers individuales se llamen entre sí.

3) Controller → Service (dependencia directa)

Ejemplo: AppointmentController ..> AppointmentService, ..> PatientService, ..> AuthService.

- Los controllers **delegan** la lógica de negocio a services:
 - AppointmentService contiene reglas: buscar conflictos, reservar sala, transacciones.
 - PatientService maneja CRUD y validaciones de paciente.
 - AuthService valida permisos/roles, maneja sesiones.

Multiplicidad y sentido: normalmente 1 controller usa 1 o varios servicios; muchos controllers pueden usar el mismo service (N → 1).

Por qué: reduce acoplamiento entre controllers y centraliza reglas de negocio reutilizables. Favorable para testing.

4) Service → Repository

Ej.: AppointmentService ..> AppointmentRepository.

- Los services usan repositorios (DAO) para persistir/leer.
- Repositorios encapsulan acceso a BD (queries, transacciones).

Por qué: separación clara entre lógica de negocio y acceso a datos. Facilita cambiar la BD sin tocar controllers.

5) EventBus (pub/sub)

- Controllers publican eventos en vez de llamar a otros controllers:
 - AppointmentController publica AppointmentCreatedEvent.
 - RecordController publica RecordUpdatedEvent.
- Servicios o listeners (NotificationService, ReportService, AuditService) **se suscriben** a esos eventos.

Por qué: desacopla componentes, permite trabajo asincrónico (notificaciones, auditoría, actualización de cache), y evita dependencias directas controller→controller.

6) Relación práctica con AuthController

- AuthController **no** debe ser llamado por AppointmentController. En su lugar:
 - AuthController maneja endpoints de login/logout.
 - AuthService es inyectado en AppointmentController para chequear permisos (hasRole, validateToken).
 - **Por qué:** evitar acoplar controladores y reutilizar el servicio de autenticación.
-

7) ReportController → ReportService → Repos

- ReportController pide reportes al ReportService, que lee de AppointmentRepository y PaymentRepository.
- **Por qué:** permite generar reportes complejos sin sobrecargar controllers.

4. Servicios transversales

- **AuthService** y **NotificationService** son utilizados por varios controllers (Auth, Appointment, Payment).

Flujo teórico: agendar cita (paso a paso)

1. Usuario (recepcionista/paciente) rellena formulario en **AppointmentView** y hace click en **Guardar**.
2. **AppointmentView** llama **AppointmentController.createAppointment(formData)**.
3. **AppointmentController**:
 - Llama a **TherapistModel.isAvailable(start,end)** y **RoomModel.isAvailable(start,end)**.
 - Llama a **AppointmentRepository.findConflicts(start,end,therapistId,roomId)** (verifica cruces).

- Si hay conflicto → devuelve error a la vista
`(AppointmentView.showError(...)).`
- Si no hay conflicto:
 - Crea `AppointmentModel` con `status = SCHEDULED`.
 - Llama `AppointmentRepository.save(appointment).`
 - Llama `NotificationService.sendEmail(patient.contact.email, ...).`
 - Devuelve éxito a la vista →
`AppointmentView.renderCalendar(updatedList).`

4. Vista actualiza UI; el paciente/terapeuta recibe notificación.

Aquí tenemos un diagrama general

